1) Reverse a String

Interview Question

How do you reverse a string in Java?

Detailed Explanation

This is one of the **most common Java interview questions**.

There are multiple ways:

- Using loop
- Using `StringBuilder`
- Using recursion
- Using streams (less common for string reverse)

Best interview answer:

Start with **StringBuilder** (simple and clean), then mention **manual loop** if asked.

Approach 1 – Using StringBuilder (Best Simple Answer)

```
public class ReverseStringExample {
    public static void main(String[] args) {
        String input = "Murali";

        String reversed = new
StringBuilder(input).reverse().toString();

        System.out.println("Original String: " + input);
        System.out.println("Reversed String: " + reversed);
    }
}
```

Output

```
Original String: Murali
Reversed String: ilaruM
```

Approach 2 – Manual Loop (Good for logic round)

```
public class ReverseStringManual {
    public static void main(String[] args) {
        String input = "Murali";
        String reversed = "";

        for (int i = input.length() - 1; i >= 0; i--) {
            reversed += input.charAt(i);
        }

        System.out.println("Reversed String: " + reversed);
    }
}
```

⚠️ Interview note: `reversed +=` creates many objects, so for large strings prefer `StringBuilder`.

Interviewer Cross-Question

Q: Which is better for performance, `StringBuilder` or `+` in loop?
A: `StringBuilder` is better because `String` is immutable and `+` in loop creates many intermediate objects.

2) Check if a String is Palindrome

Interview Question

How do you check whether a string is a palindrome in Java?

Detailed Explanation

A palindrome is a string that reads the same forward and backward.

Examples:

- `madam` ✅

- `level` ✅
- `java` ❌

Common approaches:

- Reverse string and compare
- Two-pointer approach (better logic answer)

Best interview answer:

Two-pointer is a **stronger logic answer**.

Approach 1 – Reverse and Compare

```
public class PalindromeString {
    public static void main(String[] args) {
        String input = "madam";

        String reversed = new
StringBuilder(input).reverse().toString();

        if (input.equals(reversed)) {
            System.out.println(input + " is a palindrome");
        } else {
            System.out.println(input + " is not a
palindrome");
        }
    }
}
```

Output

```
madam is a palindrome
```

Approach 2 – Two Pointer (Best Logical Answer)

```
public class PalindromeTwoPointer {
    public static void main(String[] args) {
        String input = "madam";
```

```
        boolean isPalindrome = true;

        int left = 0;
        int right = input.length() - 1;

        while (left < right) {
            if (input.charAt(left) != input.charAt(right)) {
                isPalindrome = false;
                break;
            }
            left++;
            right--;
        }

        System.out.println(input + (isPalindrome ? " is a
palindrome" : " is not a palindrome"));
    }
}
```

Interviewer Cross-Question

Q: How would you make it case-insensitive?
A: Convert to lower case before checking:

```
input = input.toLowerCase();
```

3) Check if Two Strings are Anagrams

Interview Question

How do you check if two strings are anagrams in Java?

Detailed Explanation

Two strings are anagrams if they contain the **same characters with the same frequency**, but in a different order.

Examples:

- `listen` and `silent` ✅

- `race` and `care` ✅
- `java` and `avaj` ✅
- `hello` and `world` ❌

Common approaches:

- Sort both strings and compare
- Use frequency map / array

Best simple interview answer:

Sort and compare.

Approach 1 – Sort and Compare

```
import java.util.Arrays;

public class AnagramExample {
    public static void main(String[] args) {
        String s1 = "listen";
        String s2 = "silent";

        char[] arr1 = s1.toLowerCase().toCharArray();
        char[] arr2 = s2.toLowerCase().toCharArray();

        Arrays.sort(arr1);
        Arrays.sort(arr2);

        if (Arrays.equals(arr1, arr2)) {
            System.out.println("Strings are anagrams");
        } else {
            System.out.println("Strings are not anagrams");
        }
    }
}
```

Output

```
Strings are anagrams
```

Approach 2 – Frequency Count (Better for optimization discussion)

```
public class AnagramUsingCount {
    public static void main(String[] args) {
        String s1 = "listen";
        String s2 = "silent";

        if (s1.length() != s2.length()) {
            System.out.println("Not anagrams");
            return;
        }

        int[] count = new int[256];

        for (char c : s1.toCharArray()) {
            count[c]++;
        }

        for (char c : s2.toCharArray()) {
            count[c]--;
        }

        boolean isAnagram = true;
        for (int i : count) {
            if (i != 0) {
                isAnagram = false;
                break;
            }
        }

        System.out.println(isAnagram ? "Strings are anagrams"
: "Strings are not anagrams");
    }
}
```

Interviewer Cross-Question

Q: Which approach is better — sorting or counting?
A: Counting is generally better in time complexity (`O(n)`), while sorting is `O(n log n)`.

4) Find the First Non-Repeated Character in a String

Interview Question

How do you find the first non-repeated character in a string?

Detailed Explanation

This is a **very famous interview question**.

Logic:

1. Count frequency of each character
2. Preserve insertion order
3. Find the first character with count = 1

Important:

Use **`LinkedHashMap`** because order matters.

Map-Based Solution (Best Interview Answer)

```
import java.util.LinkedHashMap;
import java.util.Map;

public class FirstNonRepeatedChar {
    public static void main(String[] args) {
        String input = "swiss";

        Map<Character, Integer> freqMap = new
LinkedHashMap<>();

        for (char ch : input.toCharArray()) {
            freqMap.put(ch, freqMap.getOrDefault(ch, 0) + 1);
        }

        for (Map.Entry<Character, Integer> entry :
freqMap.entrySet()) {
            if (entry.getValue() == 1) {
```

```
                System.out.println("First non-repeated
character: " + entry.getKey());
                break;
            }
        }
    }
}
```

Output

```
First non-repeated character: w
```

Interviewer Cross-Question

Q: Why not use `HashMap` here?
A: `HashMap` does not preserve insertion order, but `LinkedHashMap` does.

5) Find Duplicate Characters in a String

Interview Question

How do you find duplicate characters in a string?

Detailed Explanation

A duplicate character means:

- Character frequency > 1

Logic:

1. Count frequency of each character
2. Print characters whose count is more than 1

Code

```
import java.util.HashMap;
import java.util.Map;
```

```
public class DuplicateCharacters {
    public static void main(String[] args) {
        String input = "programming";

        Map<Character, Integer> freqMap = new HashMap<>();

        for (char ch : input.toCharArray()) {
            freqMap.put(ch, freqMap.getOrDefault(ch, 0) + 1);
        }

        System.out.println("Duplicate characters:");
        for (Map.Entry<Character, Integer> entry :
freqMap.entrySet()) {
            if (entry.getValue() > 1) {
                System.out.println(entry.getKey() + " -> " +
entry.getValue());
            }
        }
    }
}
```

Output

```
Duplicate characters:
r -> 2
g -> 2
m -> 2
```

Interviewer Cross-Question

Q: How do you ignore spaces and case sensitivity?
A: Normalize first:

```
input = input.toLowerCase().replace(" ", "");
```

6) Swap Two Numbers Without Third Variable

Interview Question

How do you swap two numbers without using a third variable in Java?

Detailed Explanation

This is a classic **core Java logical question**.

Common approaches:

1. Arithmetic (+, –)
2. XOR bitwise (advanced)

Best simple answer:

Use arithmetic, but mention **overflow risk**.

Approach 1 – Arithmetic

```
public class SwapNumbers {
    public static void main(String[] args) {
        int a = 10;
        int b = 20;

        System.out.println("Before swap: a = " + a + ", b = "
+ b);

        a = a + b; // 30
        b = a - b; // 10
        a = a - b; // 20

        System.out.println("After swap: a = " + a + ", b = "
+ b);
    }
}
```

Output

```
Before swap: a = 10, b = 20
After swap: a = 20, b = 10
```

Approach 2 – XOR (Advanced / Interview bonus)

```
public class SwapNumbersXOR {
    public static void main(String[] args) {
        int a = 10;
        int b = 20;

        a = a ^ b;
        b = a ^ b;
        a = a ^ b;

        System.out.println("After swap: a = " + a + ", b = "
+ b);
    }
}
```

Interviewer Cross-Question

Q: What is the drawback of arithmetic swap?
A: It may cause **integer overflow** for very large values.

7) Famous Map Program – Count Frequency of Each Character in a String

Interview Question

How do you count the frequency of each character in a string using Map?

Detailed Explanation

This is one of the **most famous map-based interview programs**.

It tests:

- `HashMap`
- `getOrDefault()`
- character iteration
- frequency logic

Logic:

1. Iterate over each character
2. If already present, increment count
3. Otherwise insert with count = 1

Code

```
import java.util.HashMap;
import java.util.Map;

public class CharacterFrequency {
    public static void main(String[] args) {
        String input = "banana";

        Map<Character, Integer> freqMap = new HashMap<>();

        for (char ch : input.toCharArray()) {
            freqMap.put(ch, freqMap.getOrDefault(ch, 0) + 1);
        }

        System.out.println("Character frequency: " +
freqMap);
    }
}
```

Output

```
Character frequency: {a=3, b=1, n=2}
```

Interviewer Cross-Question

Q: How is this different from finding duplicate characters?
A: This prints frequency of **all characters**, while duplicate-char logic filters only entries where count > 1.

Reverse Each Word in a Sentence

Interview Question

How do you reverse each word in a sentence in Java without changing the word order?

Detailed Explanation

This is a very common **String interview question**.

Example:

Input:

```
Java is powerful
```

Output:

```
avaJ si lufrewop
```

Important:

- We reverse **each individual word**
- But the **word positions remain the same**

Logic:

1. Split the sentence into words using space
2. Reverse each word
3. Join them back with spaces

Best interview answer:

- Show **loop-based solution** first
- Then mention `StringBuilder.reverse()` as a cleaner way

Approach 1 – Using StringBuilder (Best Simple Answer)

```
public class ReverseEachWord {
    public static void main(String[] args) {
        String input = "Java is powerful";
        String[] words = input.split(" ");

        StringBuilder result = new StringBuilder();
```

```
        for (String word : words) {
            result.append(new
StringBuilder(word).reverse()).append(" ");
        }

        System.out.println("Output: " +
result.toString().trim());
    }
}
```

Output

```
Output: avaJ si lufrewop
```

Approach 2 – Manual Logic (Good for logical round)

```
public class ReverseEachWordManual {
    public static void main(String[] args) {
        String input = "Java is powerful";
        String[] words = input.split(" ");

        StringBuilder finalResult = new StringBuilder();

        for (String word : words) {
            String reversedWord = "";

            for (int i = word.length() - 1; i >= 0; i--) {
                reversedWord += word.charAt(i);
            }

            finalResult.append(reversedWord).append(" ");
        }

        System.out.println("Output: " +
finalResult.toString().trim());
    }
}
```

⚠️ Interview note: `reversedWord +=` is less efficient because `String` is immutable.

Simple Explanation

This logic means:

"Break the sentence into words, reverse each word separately, and combine them back."

Interviewer Cross-Question

Q: How is this different from reversing the full sentence?
A:

- **Reverse full sentence** → entire character sequence reverses
- **Reverse each word** → only individual words reverse, word order stays same

Example:

```
Input:  Java is powerful
Full reverse: lufrewop si avaJ
Each word reverse: avaJ si lufrewop
```

2) Remove Duplicate Characters from a String

Interview Question

How do you remove duplicate characters from a string in Java while preserving the original order?

Detailed Explanation

This is a very famous **String + Collection interview question**.

Example:

Input:

```
programming
```

Output:

```
progamin
```

Important requirement:

We usually want to:

- Keep the **first occurrence**
- Remove repeated occurrences later
- Preserve insertion order

Best logic:

Use a **LinkedHashSet**

- `Set` removes duplicates
- `LinkedHashSet` preserves insertion order

Approach 1 – Using LinkedHashSet (Best Interview Answer)

```
import java.util.LinkedHashSet;
import java.util.Set;

public class RemoveDuplicateCharacters {
    public static void main(String[] args) {
        String input = "programming";

        Set<Character> uniqueChars = new LinkedHashSet<>();

        for (char ch : input.toCharArray()) {
            uniqueChars.add(ch);
        }

        StringBuilder result = new StringBuilder();
        for (char ch : uniqueChars) {
            result.append(ch);
        }

        System.out.println("Output: " + result);
    }
}
```

Output

```
Output: progamin
```

Approach 2 – Using StringBuilder + contains() (Simple but less optimal)

```
public class RemoveDuplicateCharactersSimple {
    public static void main(String[] args) {
        String input = "programming";
        StringBuilder result = new StringBuilder();

        for (char ch : input.toCharArray()) {
            if (result.indexOf(String.valueOf(ch)) == -1) {
                result.append(ch);
            }
        }

        System.out.println("Output: " + result);
    }
}
```

Simple Explanation

This logic means:

"Keep only the first time each character appears, ignore the rest."

Interviewer Cross-Question

Q: Why use `LinkedHashSet` instead of `HashSet`?
A:

- `HashSet` removes duplicates but does **not preserve order**
- `LinkedHashSet` removes duplicates **and preserves insertion order**

🔥 This is a **very common interview follow-up**.

🔥 Bonus – Remove Duplicate Words from a Sentence

Sometimes interviewer twists the question:

“Remove duplicate **words** instead of characters.”

Example:

Input:

```
java is java powerful is fun
```

Output:

```
java is powerful fun
```

Code:

```
import java.util.LinkedHashSet;
import java.util.Set;

public class RemoveDuplicateWords {
    public static void main(String[] args) {
        String input = "java is java powerful is fun";
        String[] words = input.split(" ");

        Set<String> uniqueWords = new LinkedHashSet<>();

        for (String word : words) {
            uniqueWords.add(word);
        }

        String result = String.join(" ", uniqueWords);

        System.out.println("Output: " + result);
    }
}
```

Quick Revision (Interview Short Form)

1) Reverse each word in a sentence

```
String[] words = input.split(" ");
for (String word : words) {
    result.append(new StringBuilder(word).reverse()).append("
");
}
```

2) Remove duplicate characters from string

```
Set<Character> set = new LinkedHashSet<>();
for (char ch : input.toCharArray()) {
    set.add(ch);
}
```